

❏ AI활용프로그래밍 Week 13 — 스레드 & 미니게임 (threading 따라하기)

- 이 노트북은 **PPT 예시 코드**를 Jupyter에서 **바로 실행**할 수 있게 정리한 버전입니다.
- 일부 예제(특히 `daemon=True` 무한 루프)는 Jupyter에서 **커널이 계속 살아있어** 종료가 안 될 수 있어, 안전하게 끝나도록 `Event`를 함께 사용했습니다.
- 입력이 필요한 셀(`input()`)이 있으니 `Run All` 보다는 **Step별 실행**을 권장합니다.

학습 목표

- 동시성(concurrency)과 스레드(thread)의 기본 개념을 이해한다.
- `threading.Thread`로 스레드를 생성/실행(`start`)하고 `join`으로 동기화할 수 있다.
- 공유 데이터의 경쟁 상태(race condition)를 이해하고 `Lock/Event/Queue`로 안전하게 만든다.
- 간단한 미니게임에 타이머/애니메이션 등 동시 실행 요소를 적용한다.

❏ Step 0. 실행 환경 준비

아래 셀을 실행해서 Python 버전을 확인하세요.

```
import sys
import threading
import time
from queue import Queue

print('python', sys.version.split()[0])
print('threading module:', threading.__name__)
```

동시성(Concurrency) vs 병렬성(Parallelism)

```
# 동시성: 번갈아가며 처리(논리적 동시에)
# 병렬성: 실제로 동시에 처리(멀티 코어)
# 파이썬 thread는 I/O 작업에 특히 유용
```

- I/O(입출력) 대기 중에도 다른 일을 할 수 있다.
- CPU 계산을 '진짜 병렬'로 만들려면 다른 접근이 필요할 수 있다.
- 오늘은 `threading` 기초 사용법에 집중한다.

❏ Step 1. 동기 실행의 문제: `sleep`이 전체를 멈춘다

해야 할 것

- 아래 코드를 그대로 실행
- `sleep(3)` 동안 입력이 '안 나오는' 느낌 확인

체크

- 3초 동안 아무 것도 못 하고 기다린 뒤에야 입력을 받는다.

```
import time

print('시작')
time.sleep(3)
x = input('입력: ')
print('끝', x)
```

❏ Step 2. `threading.Thread` 기본(start/join)

해야 할 것

- `work()` 함수를 만든다

- `Thread(target=work)` 생성
- `start()` 후 `join()`으로 종료 기다림

체크

- 작업 출력 후 메인 종료 가 출력된다(순서 확인).

```
import threading

def work():
    print('작업')

t = threading.Thread(target=work)
t.start()
t.join()
print('메인 종료')
```

▼ Step 3. daemon 스레드(메인이 끝나면 같이 종료)

PPT 예시는 `while True` + `daemon=True` 입니다.

⚠ **주의(Jupyter):** 노트북에서는 메인 프로그램이 끝나도 커널이 계속 살아 있어서, `daemon=True`라도 스레드가 계속 돌 수 있습니다. 그래서 아래 코드는 **PPT 의도(daemon=True)**를 유지하면서, `Event`로 안전하게 종료합니다.

```
import threading, time

stop_evt = threading.Event()

def bg():
    while not stop_evt.is_set():
        print('tick')
        time.sleep(0.5)

t = threading.Thread(target=bg, daemon=True)
t.start()
time.sleep(2)
stop_evt.set() # Jupyter에서는 안전 종료를 위해 stop 신호를 보냄
t.join(timeout=1) # daemon이라 join이 필수는 아니지만, 깔끔한 종료를 위해 잠깐 기다림
print('메인 끝')
```

▼ Step 4. race condition 재현(공유 변수)

해야 할 것

- 코드를 여러 번 실행해 본다
- `count`가 기대보다 작게 나올 수 있음 확인

체크

- 실행마다 결과가 달라지거나(특히 작게) `200000`이 아닐 수 있다.

아래는 PPT의 기본 예시입니다.

```
import threading

count = 0

def inc():
    global count
    for _ in range(100000):
        count += 1

threads = [threading.Thread(target=inc) for _ in range(2)]
for t in threads:
    t.start()
for t in threads:
    t.join()

print('count =', count) # 기대: 200000
```

✓ (선택) race를 더 잘 재현하는 버전

환경에 따라 위 코드가 항상 200000이 나올 수도 있습니다. 아래는 읽기-수정-쓰기 과정을 분리해 race가 더 잘 보이도록 만든 버전입니다.

```
import threading, time

count = 0

def inc_unstable():
    global count
    for i in range(200000):
        # 읽기-수정-쓰기 분리(원자적이지 않음) → race가 더 잘 드러남
        tmp = count
        tmp += 1
        count = tmp
        if i % 50000 == 0:
            time.sleep(0) # 다른 스레드에게 실행 기회 양보

threads = [threading.Thread(target=inc_unstable) for _ in range(4)]
for t in threads:
    t.start()
for t in threads:
    t.join()

print('count =', count, '(기대:', 200000*4, ')')
```

✓ Step 5. Lock으로 임계구역 보호

해야 할 것

- Lock을 1개 만든다
- with lock: 안에서만 count 수정

체크

- 항상 기대값이 나온다.

```
import threading

count = 0
lock = threading.Lock()

def inc():
    global count
    for _ in range(100000):
        with lock:
            count += 1

threads = [threading.Thread(target=inc) for _ in range(2)]
for t in threads:
    t.start()
for t in threads:
    t.join()

print('count =', count) # 기대: 200000
```

✓ Step 6. Queue로 안전한 통신(생산자-소비자)

해야 할 것

- 작업 스레드가 q.put()로 결과 전달
- 메인이 q.get()으로 받기

체크

- 메인에서 받은 메시지: done이 출력된다.

```
import threading
from queue import Queue

q = Queue()
```

```
def worker():
    # 어떤 계산 결과를 만든다고 가정
    q.put('done')

t = threading.Thread(target=worker)
t.start()
msg = q.get()
t.join()
print('메인에서 받은 메시지:', msg)
```

Timer/주기 작업: `threading.Timer`

- `threading.Timer`는 일정 시간 뒤 함수 실행(1회)입니다.
- 반복 타이머는 루프+`sleep` 또는 스케줄링이 필요합니다.

```
import threading

def say():
    print('tick (Timer)')

t = threading.Timer(1.0, say)
t.start()
t.join() # Timer 쓰레드가 끝날 때까지 기다림
print('Timer 종료')
```

Step 7. `Event`로 종료 신호 보내기

해야 할 것

- `evt = threading.Event()` 만들기
- 백그라운드 쓰레드에서 `evt.is_set()` 확인
- 메인에서 `evt.set()`으로 종료

체크

- tick이 몇 번 출력된 뒤 (종료)가 출력되고 깔끔히 끝난다.

```
import threading, time

evt = threading.Event()

def ticker():
    while not evt.is_set():
        print('tick')
        time.sleep(0.5)

t = threading.Thread(target=ticker)
t.start()
time.sleep(2)
evt.set()
t.join()
print('종료')
```

예제: 로딩 스피너(백그라운드)

PPT 예시는 공유 변수 (`running=True/False`)를 사용합니다. 여기서는 더 안전하게 `Event`로 종료 신호를 보냅니다.

```
import threading, time

stop_evt = threading.Event()

def spinner():
    while not stop_evt.is_set():
        print('.', end='', flush=True)
        time.sleep(0.2)

t = threading.Thread(target=spinner, daemon=True)
t.start()
```

```
# ... 메인 작업을 하는 척 ...
time.sleep(1.5)

stop_evt.set()
t.join(timeout=1)
print('ㄴ스피너 종료')
```

▼ Step 8. Mini-game 스텔레톤: 5초 제한 입력

해야 할 것

- 타이머 쓰레드를 `daemon=True` 로 시작
- `timeout` 플래그(또는 Event)로 시간 초과 판단
- 입력 후 결과 출력

체크

- 5초 안에 입력하면 `입력 성공`, 늦으면 `시간 초과!` 가 출력된다.

⚠ 참고: 이 코드는 **입력을 강제로 끊지는 못하고**, 입력이 끝난 뒤에 '늦었는지'를 판정합니다.

```
import threading, time

timeout = False

def timer():
    global timeout
    time.sleep(5)
    timeout = True

threading.Thread(target=timer, daemon=True).start()
ans = input('5초 안에 입력: ')

if timeout:
    print('시간 초과!')
else:
    print('입력 성공:', ans)
```

게임 설계: 상태(State)로 생각하기

예시:

```
READY -> WAITING -> GO -> DONE
이벤트: timer 완료, 사용자 입력
```

- 복잡한 프로그램은 상태로 나누면 깔끔해집니다.
- 쓰레드는 상태 전환 이벤트를 발생시키는 역할을 할 수 있습니다.

안전한 설계 팁

1. 공유 변수 최소화
2. 공유가 필요하면 `Lock/Event/Queue` 사용
3. 출력/입력은 한 곳에서 관리

쓰레드 디버깅 팁

- 로그(출력)를 남겨 흐름 확인
- `join()` 으로 종료를 보장
- 데드락(서로 기다림) 주의

With AI: 쓰레드 코드 생성 프롬프트 템플릿(복사해서 사용)

- 역할: 너는 파이썬 동시성 전문가다.
- 목표: (미니게임/타이머/로딩) 기능을 쓰레드로 구현하고 싶다.

- 조건: 공유 데이터는 최소화하고, 필요하면 `Lock/Event/Queue`를 사용
- 요청:
 1. 설계 설명(상태/흐름)
 2. 코드
 3. 테스트 시나리오
 4. 위험 요소(race) 점검
- 추가: 내 코드가 있으면 버그 재현과 수정안을 제시해줘.

✓ 실습 1: 백그라운드 타이머 만들기

문제: 1초마다 'tick'을 출력하는 쓰레드를 만들고, 5초 후 종료하라. 힌트: `daemon` 또는 `Event` 사용
아래 셀은 **TODO(실습용)**입니다.

```
import threading, time

# TODO: 1초마다 tick 출력, 5초 후 종료
# 힌트: Event를 만들어서 while not evt.is_set() 루프 사용
```

✓ 정답 예시(Event 사용)

```
import threading, time

evt = threading.Event()

def tick_worker():
    while not evt.is_set():
        print('tick')
        time.sleep(1)

t = threading.Thread(target=tick_worker)
t.start()

time.sleep(5)
evt.set()
t.join()
print('종료')
```

✓ 실습 2: 디버깅 — 경쟁 상태 재현 후 Lock으로 수정

아래 코드를 여러 번 실행해 보고, `Lock`을 적용해 안정적으로 기대값이 나오게 만들어 보세요.

```
import threading

count = 0

def inc():
    global count
    for _ in range(100000):
        count += 1

threads = [threading.Thread(target=inc) for _ in range(2)]
for t in threads:
    t.start()
for t in threads:
    t.join()

print(count) # 기대: 200000

# TODO: Lock을 적용해 항상 200000이 나오게 수정
```

✓ 정답 예시(Lock 적용)

```
import threading

count = 0
lock = threading.Lock()
```

```
def inc():
    global count
    for _ in range(100000):
        with lock:
            count += 1

threads = [threading.Thread(target=inc) for _ in range(2)]
for t in threads:
    t.start()
for t in threads:
    t.join()

print(count) # 기대: 200000
```

✓ 실습 3: 리팩터링 — Queue로 안전하게

공유 변수를 직접 수정하는 대신, 작업 스레드가 만든 결과를 Queue로 전달해 보세요.

```
import threading
from queue import Queue

q = Queue()

# TODO: worker 스레드가 결과를 q.put()
# TODO: 메인이 q.get()해서 출력
```

✓ 정답 예시

```
import threading
from queue import Queue

q = Queue()

def worker():
    # 예: 계산 결과
    q.put({'result': 42})

t = threading.Thread(target=worker)
t.start()

msg = q.get()
t.join()
print('메인에서 받은 메시지:', msg)
```

연습문제(추가)

1. 스레드 2개를 만들어 각각 다른 문장을 출력하라.
2. join을 사용해 메인에서 종료를 기다려라.
3. Lock을 사용해 공유 변수(count) 증가를 안전하게 만들어라.
4. Queue를 사용해 스레드가 만든 결과를 메인으로 전달하라.

미니 퀴즈(5문항)

1. start()의 역할은?
A. 스레드 객체 생성 B. 스레드 실행 시작 C. 종료 대기
2. join()의 역할은?
A. 실행 시작 B. 결과 출력 C. 종료까지 기다림
3. race condition이란?
A. 속도 경주 B. 실행 순서에 따라 결과가 달라짐 C. 문법 오류
4. 공유 자원을 보호하는 도구는?
A. Lock B. sort C. split
5. Queue의 장점은?
A. 자동 정렬 B. thread-safe 통신 C. 파일 저장

정답: 1-B, 2-C, 3-B, 4-A, 5-B

